

BERKA BANK DATASET

Transaction-Level SQL Analytics

Query Observations, Results, and Business Insights

Urbanus Kathitu | Github: <https://github.com/KathituCodes> | June 2026

DATASET OVERVIEW

The Berka dataset contains real financial records from a Czech bank covering the period 1993 to 1998. It is structured across eight relational tables that together capture the full lifecycle of a banking relationship: from customer registration and account opening, through everyday transactions, to loan issuance and final repayment outcomes.

Dataset Source: <https://www.kaggle.com/datasets/marceloventura/the-berka-dataset?resource=download>

Table	Row Count	What it Contains
account	4,500	Every bank account. The central entity.
client	5,369	Every customer. Linked to accounts via disp.
district	77	Regional data: population, salary, crime rate.
disp	5,369	The bridge between clients and accounts.
loan	682	Every loan issued. Contains our target variable: status.
order	6,471	Standing payment orders (e.g. rent payments).
trans	1,056,320	Every single transaction. The heartbeat of the dataset.
card	892	Credit cards issued to customers.

Before writing a single analytical query, I explored the dataset to understand the projects goal and the data description: <https://webpages.charlotte.edu/mirsad/itcs6265/group1/index.html> . This is a crucial step I take before analysis because querying data you have not yet understood produces misleading results.

UNDERSTANDING THE LOAN STATUS DISTRIBUTION

The loan table contains a status column that records the outcome of every loan. I ran a **GROUP BY** query to understand how many loans fell into each category before making any assumptions about the data.

Status	Count	Meaning
C	403	Loan still running, no problems. Currently good.
A	203	Loan finished, client paid everything back on time. Good client.
D	45	Loan still running, client already in debt trouble. Currently bad.
B	31	Loan finished, client did NOT pay it all back. Bad client.

Out of 682 total loans: 606 are good or currently fine (A + C). 76 are bad or in trouble (B + D). The core credit risk question is: how do you identify the 76 bad ones before you give them money?

For my analytical queries I simplified these four statuses into two binary labels: GOOD (A or C) and BAD (B or D). This is the foundation of how credit scoring models frame the target variable in a real production environment.

UNDERSTANDING THE TRANSACTION STRUCTURE

With over one million rows, the trans table required careful exploration before analysis. The data is in Czech so I mapped the key terms to their English equivalents before writing any queries.

Czech Term	English Meaning	Transaction Count
VYDAJ + VYBER	Cash withdrawal (debit)	418,252
PREVOD NA UCET	Transfer to another account (sending money out)	208,283
PRIJEM (no operation)	Credit (Money entering account)	183,114
VKLAD	Cash deposit	156,743
PREVOD Z UCTU	Transfer from another account (receiving money in)	65,226
VYBER	Cash withdrawal (withdrawal type)	16,666
VYBER KARTOU	Credit card withdrawal	8,036

Understanding this structure was essential before writing any query that filters by transaction type. Using the wrong type value would have returned incorrect results without throwing an error.

QUERY 1: DO HIGH BALANCE ACCOUNTS DEFAULT LESS?

The Business Question

This is a question every fintech credit team asks before building a scoring model: if a customer maintains a consistently high average balance, are they less likely to default on a loan? I set out to answer this using the transaction history across all 1,056,320 records.

The Approach

I used three CTEs (Common Table Expressions) chained together. A CTE is a temporary named result that exists only for the duration of the query. Think of it as a whiteboard where I write intermediate working before arriving at the final answer.

- **CTE 1:** I calculated the average balance for every account across all their transactions.
- **CTE 2:** I used the NTILE(5) window function to split all accounts into five equal groups ranked by average balance. Group 1 is the bottom 20% with the lowest balances. Group 5 is the top 20% with the highest balances.
- **CTE 3:** I filtered to keep only the top 20% and bottom 20% for a clean comparison.
- **Final SELECT:** I joined to the loan table and calculated the default rate for each group.

The Results

Balance Tier	Total Loans	Bad Loans	Default Rate
BOTTOM 20% (lowest balances)	40	15	37.50%
TOP 20% (highest balances)	246	7	2.85%

Accounts in the bottom 20% by average balance default at a rate 13 times higher than accounts in the top 20%. This is one of the strongest signals I found in the entire dataset.

My Observations

When I look at these numbers I see two things that matter:

37.50% divided by 2.85% = 13.15 (13x gap)

- If the bank lends to someone with a consistently low average balance, 37 out of every 100 loans go bad. That is an enormous loss rate for any lending institution.
- If the bank lends to someone with a consistently high average balance, only 3 out of every 100 loans go bad. The risk is reduced by a factor of 13 simply by looking at one behavioural signal.
- The difference is not marginal. It is a 13x gap in default risk between the two groups. This tells me that average balance is a strong candidate feature for a credit scoring model.
- The top 20% group also had far more loans, 246 versus 40, which tells me the bank was already making implicit decisions based on account standing, even if not through a formal scoring model.

What This Means in a Fintech Context

Companies like Pezesha and M-KOPA use mobile money transaction data to infer account balance behaviour for customers who may not have a formal bank account. The principle is identical: consistent positive balance behaviour is a strong proxy for creditworthiness. This query demonstrates exactly the kind of feature validation that precedes model building in a real credit scoring workflow.

QUERY 2: ARE THERE ANY MONTH-OVER-MONTH TRANSACTION VELOCITY DROP OCCURANCES

Velocity drop is when an account experiences a sudden drop in transaction activity and it is one of the earliest warning signs of default risk.

The Business Question

If a customer has a loan outstanding and their transaction activity suddenly drops sharply, that is an early warning signal. It could mean they lost their job, their business collapsed, or they are deliberately avoiding using their

account. I set out to flag every account whose monthly transaction count dropped by more than 40% compared to the previous month.

The Approach

This query introduced window functions, specifically the LAG() function. LAG() looks at the previous row in a sorted sequence. In this context, for each account and each month, LAG() reaches back to the previous month and retrieves its transaction count. This allowed me to calculate month-over-month percentage change without loops or manual comparisons.

- **CTE 1:** I counted transactions per account per month, converting the YYMMDD date format to proper year and month integers.
- **CTE 2:** I used LAG() partitioned by account_id and ordered chronologically to attach the previous month's transaction count to each row.
- **CTE 3:** I calculated the percentage change between the current month and the previous month.
- **Final SELECT:** I flagged any account with a drop greater than 40% as HIGH RISK and displayed the worst drops first.

Sample Results (Top 5 Worst Drops)

Account ID	Year	Month	Current Txns	Previous Txns	% Change	Risk Flag
10857	1998	December	1	10	-90.00%	HIGH RISK
10788	1996	March	1	7	-85.71%	HIGH RISK
10019	1998	December	1	7	-85.71%	HIGH RISK
3825	1997	March	1	6	-83.33%	HIGH RISK
573	1998	February	3	18	-83.33%	HIGH RISK

My Observations

Three things stood out to me when I analysed these results:

- December 1998 appears repeatedly in the results. After investigating, I realised this is a data collection artefact, not a genuine risk signal. December 1998 is the final month in the dataset and many accounts have fewer transactions simply because the data collection ended mid-month. In a production environment I would filter out the most recent month to eliminate this noise before flagging risk.
- The accounts with the most extreme drops went from 10 or more transactions to just 1 in a single month. A drop of 90% in one month is severe enough to trigger an immediate review by a credit officer regardless of the threshold chosen.
- The LAG() window function processed all 1,056,320 transaction rows and performed month-over-month comparisons for every account in a single pass. This is fundamentally more efficient than any loop-based approach and is how production SQL systems handle this type of time-series analysis.

Recognising that December 1998 results are artefacts of data collection rather than genuine risk signals is exactly the kind of domain-aware data interpretation that separates a careful analyst from someone who reports results mechanically.

What This Means in a Fintech Context

Post-disbursement monitoring is one of the hardest problems in lending. Most credit models score a borrower at the point of application and then rely on repayment behaviour to detect problems. But by the time someone misses a payment, the damage is already done. Transaction velocity monitoring detects the behavioural precursors to default, giving a credit team time to intervene before a payment is missed.

QUERY 3: CASH-OUT BEFORE DEFAULT RISK DETECTION

The Business Question

When someone knows they cannot repay a loan, a common pattern is to withdraw as much cash as possible before they stop paying. This is called cash-out before default and it is one of the most studied fraud patterns in consumer lending. I designed a query to flag accounts that meet two conditions simultaneously: they have a bad or troubled loan, and their most recent transaction was a large outgoing payment relative to their average monthly income.

The Approach

My first implementation used a correlated subquery to find the latest transaction per account. A correlated subquery runs once for every row in the table. With 1,056,320 rows that meant over one million individual subquery executions. The query ran for over an hour without completing and I cancelled it.

We are looking for accounts that meet 2 conditions simultaneously:

- Condition 1: They have a loan that is bad or in trouble (status B or D)
- Condition 2: Their recent transaction was a cash withdrawal that was larger than 80% of their average monthly inflow

These two signals together indicate possible intentional cash out before default.

This is a real production lesson. A technically correct query that runs for an hour is not a production-ready query. I rewrote it using ROW_NUMBER(), a window function, which processes all rows in a single pass and reduced the runtime to seconds.

- **CTE 1:** I calculated the average monthly inflow per account, filtering to PRIJEM transactions only (incoming money).
- **CTE 2:** I used ROW_NUMBER() partitioned by account and ordered by date descending to rank every transaction. Rank 1 is the most recent transaction for that account.
- **CTE 3:** I filtered to only rank 1 rows, giving me one row per account representing their latest transaction.
- **CTE 4:** I identified all accounts with a bad or troubled loan (status B or D).
- **Final SELECT:** I joined all four CTEs and flagged accounts where the last transaction was an outgoing payment exceeding the threshold percentage of their average inflow.

Threshold Comparison: 80% vs 50%

I ran the query twice with different threshold settings to observe how the sensitivity of the risk flag changes. This is the same decision a credit team makes when tuning a model: how aggressive should the flagging be?

Account ID	Loan Amount	Status	Last Withdrawal	Avg Inflow	% of Inflow	@ 80% Threshold	@ 50% Threshold
10857	385,560	D	6,426.30	8,721.80	73.68%	MONITOR	HIGH RISK
2176	299,088	B	1,085.00	13,765.84	7.88%	MONITOR	MONITOR
3678	316,140	D	42.80	5,047.83	0.85%	MONITOR	MONITOR
1888	57,360	B	36.20	10,192.21	0.36%	MONITOR	MONITOR

3037	277,884	D	14.60	11,308.64	0.13%	MONITOR	MONITOR
4081	137,904	B	5.50	6,236.46	0.09%	MONITOR	MONITOR
3273	94,512	B	6.80	17,769.74	0.04%	MONITOR	MONITOR
2236	139,464	D	0.40	9,600.54	0.00%	MONITOR	MONITOR

My Observations

The threshold comparison produced the most important insight of the entire project:

- At an 80% threshold, zero accounts were flagged as HIGH RISK. Every account showed as MONITOR. A credit officer reviewing this output might conclude there is no cash-out risk in the portfolio.
- At a **50% threshold**, account **10857** immediately emerged as **HIGH RISK**. This account carries the largest troubled loan in the bad loan group at **385,560**, has a **loan status of D** meaning it is still running but the client is already in trouble, and their last transaction was a transfer of **6,426** to another account, which is **73.68%** of their average monthly inflow. The fact that it was a transfer to another account rather than a simple cash withdrawal is particularly significant. It suggests deliberate movement of funds rather than routine spending.
- The remaining seven accounts all show withdrawal amounts **below 8%** of their average inflow. These are routine small transactions and correctly remain as **MONITOR** regardless of the threshold chosen.

Account 10857 is the only genuine risk signal in this dataset under the cash-out before default pattern definition. It has the largest loan, the worst status, and the most significant recent outflow relative to income. At an 80% threshold this signal is missed entirely. At 50% it surfaces immediately.

The Precision vs Recall Tradeoff

This threshold experiment demonstrates one of the most fundamental tensions in credit risk modelling:

- A high threshold catches fewer accounts but the ones it catches are almost certainly genuine risks. Fewer false positives, but more real risks slipping through.
- A low threshold catches more accounts but also flags good customers incorrectly. More genuine risks caught, but more investigation resources wasted on false alarms.
- There is no mathematically correct threshold. The right number depends on how much the business values catching bad loans versus the cost of inconveniencing good customers. A credit team sets it based on how much risk they are willing to accept. That is a business decision a data scientist informs but does not make alone.

In practice I would present both versions of this output to a credit team and let them decide where to set the threshold based on their risk appetite and the cost of each type of error.

That concludes my analysis of the Berka Bank Dataset using SQL.

SUMMARY OF INSIGHTS ACROSS ALL THREE QUERIES

Query	Business Question	Key Finding	Production Application
1: Balance Tier vs Default	Do high balance accounts default less?	13x difference in default rate between top and bottom 20%	Average balance is a strong candidate feature for a credit scorecard

2: Velocity Drop Detection	Does a drop in transactional activity signal default risk?	Some accounts drop 90% in a single month. December 1998 results are artefacts.	Post-disbursement monitoring to catch pre-default behaviour early
3: Cash-Out Before Default	Can we detect fund withdrawal before default?	Account 10857 is the only genuine signal. Threshold tuning changes who gets flagged.	Fraud detection layer on top of standard credit scoring

Taken together these three queries demonstrate the full analytical workflow of a fintech data science team: feature validation, behavioural monitoring, and fraud signal detection. Each query uses a different SQL technique: CTEs with NTILE for Query 1, CTEs with LAG for Query 2, and CTEs with ROW_NUMBER for Query 3. Together they form a complete portfolio of advanced SQL applied to a real financial dataset.